📚 DICIONÁRIOS DE FREQUÊNCIA E AGRUPAMENTOS

🎯 REVISÃO RÁPIDA: Cruzamento de Dicionários

Antes de começarmos, vamos lembrar como **cruzar informações** de dicionários diferentes!**ruzamento de Dicionários**

👣 PASSO A PASSO: Cruzamento de Dicionários

```
# Temos dois dicionários:
dProdTipo = {
    'álcool Gel': 'higiene',
    'água': 'bebida',
    'chocolate': 'bebida'
}

dTipoDep = {
    'higiene': 'z1',
    'bebida': 'z2',
    'alimento': 'z3'
}

# Queremos: produto → depósito
dProdDep = {}

# PASSO 1: Percorrer cada produto
for produto, tipo in dProdTipo.items():
    print(f"➡️ Produto: {produto} | Tipo: {tipo}")

    # PASSO 2: Acessar o depósito deste tipo
    deposito = dTipoDep.get(tipo)
    print(f"   🔍 Acessando depósito do tipo '{tipo}': {deposito}")

    # PASSO 3: Guardar no novo dicionário
    dProdDep[produto] = deposito
    print(f"   ✅ Salvo: {produto} → {deposito}")
    print("-" * 30)

print("\n📌 RESULTADO FINAL:")
print(dProdDep)
```

SAÍDA:

```
➡️ Produto: álcool Gel | Tipo: higiene
   🔍 Acessando depósito do tipo 'higiene': z1
   ✅ Salvo: álcool Gel → z1
-----------------------------
➡️ Produto: água | Tipo: bebida
   🔍 Acessando depósito do tipo 'bebida': z2
   ✅ Salvo: água → z2
-----------------------------
➡️ Produto: chocolate | Tipo: bebida
   🔍 Acessando depósito do tipo 'bebida': z2
   ✅ Salvo: chocolate → z2
-----------------------------

📌 RESULTADO FINAL:
{'álcool Gel': 'z1', 'água': 'z2', 'chocolate': 'z2'}
```

PARTE 1: DICIONÁRIO DE FREQUÊNCIAS

🤔 **O que é?** É um dicionário que **conta quantas vezes** cada coisa aparece!

Exemplos da vida real:

- 🏢 Contar quantos produtos tem em cada depósito
- 👥 Contar quantos alunos tem em cada turma
- 🍎 Contar quantas frutas de cada tipo

🎨 PRODUTOS COM SEUS DEPÓSITOS→Dicionário de Frequências

Produto	Depósito
álcool Gel	z1
álcool 90	z1
água	z2
refrigerante	z2
chocolate	z2
desinfetante	z1
sabonete	z1
cerveja	z2
água sanitária	z1
biscoito	z3
café	z3
leite	z3

CONTAGEM POR DEPÓSITO	
z1 → 5 produtos	← 5 produtos no z1
z2 → 4 produtos	← 4 produtos no z2
z3 → 3 produtos	← 3 produtos no z3

A LÓGICA: Como pensar?

```
Para CADA produto com seu depósito:

    ❓ PERGUNTA: Este depósito já foi contado antes
(já apareceu antes)?

    ├─▶ SIM → Obtém o valor atual e soma +1
    │
    └─▶ NÃO → Cria uma entrada nova com valor = 1
```

👣 PASSO A PASSO COM GET

```python
def contar_produtos_por_deposito(dProdDep):
    """
    Função que conta quantos produtos tem em cada depósito
    """

    # PASSO 0: Criar um dicionário VAZIO para guardar as contagens
    dFreq = {}

    print("🔍 INICIANDO CONTAGEM...\n")

    # PASSO 1: Percorrer cada produto e seu depósito
    for (produto, deposito) in dProdDep.items():
        print(f"➡️ Analisando: {produto} está no depósito {deposito}")

        # PASSO 2: Verificar se este depósito JÁ EXISTE no dicionário
        # O método .get(deposito, 0) tenta obter o valor atual
        # Se não existir, retorna 0, caso contrário, retorna o valor atual
        contagem_atual = dFreq.get(deposito, 0)

        print(f"   📊 Contagem atual do depósito {deposito}: {contagem_atual}")

        # PASSO 3: Calcular a NOVA contagem (soma +1)
        nova_contagem = contagem_atual + 1

        print(f"   ➕ Somando +1: {contagem_atual} + 1 = {nova_contagem}")

        # PASSO 4: Atualizar o dicionário com a nova contagem
        dFreq[deposito] = nova_contagem

        print(f"   ✅ Depósito {deposito} agora tem {nova_contagem} produto(s)")
        print("-" * 50)

    return dFreq

# Vamos testar!
dProdDep = {
    'álcool Gel': 'z1',
    'álcool 90': 'z1',
    'água': 'z2',
    'refrigerante': 'z2',
    'chocolate': 'z2',
    'desinfetante': 'z1',
    'sabonete': 'z1',
    'cerveja': 'z2',
    'água sanitária': 'z1',
    'biscoito': 'z3',
    'café': 'z3',
    'leite': 'z3'
}

resultado = contar_produtos_por_deposito(dProdDep)
print("\n🎯 RESULTADO FINAL:")
print(f"dFreq = {resultado}")
```

🎬 EXECUÇÃO PASSO A PASSO

Vamos acompanhar como o dicionário vai se transformando:

```
ITERAÇÃO 1: Produto 'álcool Gel' no depósito 'z1'
```

```
dFreq.get('z1', 0) → 0 (não existe)
dFreq['z1'] = 0 + 1 = 1
dFreq = {'z1': 1}
```

```
ITERAÇÃO 2: Produto 'álcool 90' no depósito 'z1'
```

```
dFreq.get('z1', 0) → 1 (já existe)
dFreq['z1'] = 1 + 1 = 2
dFreq = {'z1': 2}
```

```
ITERAÇÃO 3: Produto 'água' no depósito 'z2'
```

```
dFreq.get('z2', 0) → 0 (não existe)
dFreq['z2'] = 0 + 1 = 1
dFreq = {'z1': 2, 'z2': 1}
```

```
... e assim por diante
```

💡 VERSÃO MAIS CURTA (Mas mesmo raciocínio)

```python
def contar_produtos_por_deposito(dProdDep):
    dFreq = {}
    for (produto, deposito) in dProdDep.items():
        # Tudo em uma linha!
        dFreq[deposito] = dFreq.get(deposito, 0) + 1
    return dFreq

# É a MESMA coisa! Mais curto, mas faz exatamente o mesmo:
# 1. Obtém o valor atual (ou 0)
# 2. Soma 1
# 3. Guarda de volta
```

🔬 **MINITESTE 1: Dicionário de Frequências**

Questão 1

Complete o código para contar quantas pessoas tem cada sobrenome:

```python
pessoas = ['Ana Silva', 'João Souza', 'Maria Silva', 'Pedro Santos', 'Carlos Souza']
dFreq = {}
for pessoa in pessoas:
    # Complete aqui
    # Dica: use pessoa.split() para separar nome e sobrenome
    pass
```

Questão 2

Se temos `dFreq = {'z1': 5, 'z2': 4, 'z3': 3}`, o que significa?

- a) z1, z2 e z3 são produtos
- b) Existem 5 produtos no z1, 4 no z2 e 3 no z3
- c) Existem 5 depósitos no total

📋 PARTE 2: DICIONÁRIO DE AGRUPAMENTOS

🤔 **O que é?** É um dicionário que **agrupa itens de mesma categoria em listas**!

Exemplos da vida real:

- 🏬 Quais produtos estão em cada depósito?
- 👥 Lista de alunos por turma
- 🛒 Contar quantas frutas de cada tipo

🎨 PRODUTOS COM SEUS DEPÓSITOS→Dicionário de AGRUPAMENTO POR DEPÓSITO

Produto	Depósito
álcool Gel	z1
álcool 90	z1
água	z2
refrigerante	z2
chocolate	z2
desinfetante	z1
sabonete	z1
cerveja	z2
água sanitária	z1
biscoito	z2
café	z2
leite	z3

AGRUPAMENTO POR DEPÓSITO

z1 → [álcool Gel', 'álcool 90', desinfetante', 'sabonete', 'água sanitária']

z2 → [água', 'refrigerante', 'chocolate', 'cerveja']

z3 → [[biscoito', 'café', 'leite']

A LÓGICA: Como pensar?

```
Para CADA produto com seu depósito:

    ❓ PERGUNTA: Este depósito já tem uma lista (já
apareceu antes)?

    ├─▶ SIM → Obtém a lista atual e inclui o produto
    │
    └─▶ NÃO → Cria uma lista nova com este produto e
              inclui um novo depósito no dicionário
```

🎯 PASSO A PASSO COM GET

```python
def agrupar_com_get_CORRETO(dProdDep):
    dGrupos = {}

    for produto, deposito in dProdDep.items():
        print(f"\n➡️ Processando: {produto} → {deposito}")

        # PASSO 1: Pegar lista atual (ou [] se não existir)
        lista = dGrupos.get(deposito, [])
        print(f"   📋 Lista obtida: {lista}")

        # PASSO 2: Adicionar o produto
        lista.append(produto)
        print(f"   ➕ Após append: {lista}")

        # PASSO 3: ATUALIZAR o dicionário (MUITO IMPORTANTE!)
        #IMPORTANTE: O  get retorna uma CÓPIA da lista (ou uma nova), mas não atualiza o
        #             dicionário automaticamente!
        dGrupos[deposito] = lista
        print(f"   ✅ dGrupos atualizado: {dGrupos}")
```

```
    return dGrupos

resultado_correto = agrupar_com_get_CORRETO(dProdDep)
print("\n🎯 RESULTADO CORRETO:", resultado_correto)
```

SAÍDA:

```text
🔄 Processando: álcool Gel → z1
   📋 Lista obtida: []
   ➕ Após append: ['álcool Gel']
   ✅ dGrupos atualizado: {'z1': ['álcool Gel']}

🔄 Processando: álcool 90 → z1
   📋 Lista obtida: ['álcool Gel']
   ➕ Após append: ['álcool Gel', 'álcool 90']
   ✅ dGrupos atualizado: {'z1': ['álcool Gel', 'álcool 90']}

🔄 Processando: água → z2
   📋 Lista obtida: []
   ➕ Após append: ['água']
   ✅ dGrupos atualizado: {'z1': ['álcool Gel', 'álcool 90'], 'z2': ['água']}

🎯 RESULTADO CORRETO: {'z1': ['álcool Gel', 'álcool 90'], 'z2': ['água']}
```

🎬 EXECUÇÃO PASSO A PASSO

Vamos ver a evolução do dicionário:

```
ITERAÇÃO 1: Produto 'álcool Gel' no depósito 'z1'
```

```
dGrupos.get('z1', []) → [] (lista vazia)
[].append('álcool Gel') → ['álcool Gel']
dGrupos['z1'] = ['álcool Gel']
dGrupos = {'z1': ['álcool Gel']}
```

```
ITERAÇÃO 2: Produto 'álcool 90' no depósito 'z1'
```

```
dGrupos.get('z1', []) → ['álcool Gel']
['álcool Gel'].append('álcool 90')
→ ['álcool Gel', 'álcool 90']
dGrupos['z1'] = ['álcool Gel', 'álcool 90']
dGrupos = {'z1': ['álcool Gel', 'álcool 90']}
```

```
ITERAÇÃO 3: Produto 'água' no depósito 'z2'
```

```
dGrupos.get('z2', []) → []
[].append('água') → ['água']
dGrupos['z2'] = ['água']
dGrupos = {
  'z1': ['álcool Gel', 'álcool 90'],
  'z2': ['água']
}
```

💡 VERSÃO MAIS CURTA (Mas mesmo raciocínio)

```
def agrupar_produtos_por_deposito(dProdDep):
```

```python
    dGrupos = {}
    for produto, deposito in dProdDep.items():
        # Obtem lista atual (ou cria nova) e adiciona. Não eficiente porque o + cria uma cópia
        dGrupos[deposito] = dGrupos.get(deposito, []) + [produto]
    return dGrupos

# OU usando append (mais eficiente)
def agrupar_produtos_por_deposito(dProdDep):
    dGrupos = {}
    for produto, deposito in dProdDep.items():
        if deposito not in dGrupos:
            dGrupos[deposito] = []
        dGrupos[deposito].append(produto)
    return dGrupos
```

🎁 PARTE 3: MÉTODO `setdefault()` - O ATALHO MÁGICO!

🤔 Sempre que fazemos agrupamento, precisamos:

1. Verificar se a chave existe
2. Se não existir, criar uma lista vazia
3. Adicionar o item na lista
4. Atualizar o dicionário

O `setdefault()` faz tudo isso em UM PASSO!

🎨 setdefault vs método tradicional

👣 PASSO A PASSO: Entendendo setdefault

```python
# O que setdefault FAZ:
# 1. Acessa a chave
# 2. Se NÃO existir, cria com o valor padrão
# 3. RETORNA o valor (existente ou o novo)

# Exemplo 1: Chave NÃO existe
d = {}
print("Antes:", d)

# setdefault vai CRIAR a chave 'z1' com lista vazia e RETORNAR a lista
lista = d.setdefault('z1', [])
print("Depois de setdefault:", d)
print("Lista retornada:", lista)

# Agora podemos usar a lista diretamente!
lista.append('álcool Gel')
print("Depois de append:", d)

print("-" * 40)

# Exemplo 2: Chave JÁ existe
d2 = {'z2': ['água']}
print("Antes:", d2)

# setdefault vai RETORNAR a lista existente (não cria nova)
lista2 = d2.setdefault('z2', [])
print("Lista retornada:", lista2)
lista2.append('refrigerante')
print("Depois:", d2)
```

SAÍDA:

```
Antes: {}
Depois de setdefault: {'z1': []}
Lista retornada: []
Depois de append: {'z1': ['álcool Gel']}
----------------------------------------
Antes: {'z2': ['água']}
Lista retornada: ['água']
Depois: {'z2': ['água', 'refrigerante']}
```

✨ Agrupamento com setdefault (mais elegante!)

```python
def agrupar_com_setdefault(dProdDep):
    dGrupos = {}

    for produto, deposito in dProdDep.items():
        print(f"➡️ Processando: {produto} → {deposito}")

        # setdefault FAZ TUDO:
        # - Verifica se depósito existe
        # - Se não existir, cria com []
        # - Retorna a lista (existente ou nova)
        # - Já podemos dar append direto!
        dGrupos.setdefault(deposito, []).append(produto)

        print(f"   dGrupos agora: {dGrupos}")
        print("-" * 30)

    return dGrupos
```

```python
resultado = agrupar_com_setdefault(dProdDep)
print("\n🎯 RESULTADO FINAL:")
for dep, produtos in resultado.items():
    print(f"   {dep}: {produtos}")
```

🧪 MINITESTE 2: Dicionário de Agrupamentos

Questão 1

Complete o código para agrupar pessoas por sobrenome:

```python
pessoas = ['Ana Silva', 'João Souza', 'Maria Silva', 'Pedro Santos', 'Carlos Souza']

dGrupos = {}

for pessoa in pessoas:
    nome_completo = pessoa.split()
    sobrenome = nome_completo[1]  # Obtém o sobrenome

    # Complete aqui
    # Use setdefault para agrupar
    Pass
```

Questão 2

O que faz o código abaixo?

```python
d = {}
d.setdefault('grupo1', []).append('item1')
d.setdefault('grupo1', []).append('item2')
d.setdefault('grupo2', []).append('item3')
```

- a) Cria um erro
- b) Cria `{'grupo1': ['item1', 'item2'], 'grupo2': ['item3']}`
- c) Cria `{'grupo1': 'item1', 'grupo2': 'item3'}`

🔄 PARTE 4: DICIONÁRIO DE DICIONÁRIOS

🤔 **O que é?** É quando o **VALOR** de um dicionário éOUTRO DICIONÁRIO!

Exemplo da vida real:

- Cada produto tem um dicionário com várias informações (tipo, depósito, preço)

🎨 Dicionário de Dicionários

```
DICIONÁRIO EXTERNO (chave = produto)
╔═══════════════════════════════════════════════════════╗
║                                                       ╕
│  'álcool Gel' →   ╔═══════════════════════════╗       │
│                   ║ DICIONÁRIO INTERNO        ╕       │
│                   ╞═══════════════════════════╡       │
│                   │ 'tipo': 'higiene'         │       │
│                   │ 'dep': 'z1'               │       │
│                   ╘═══════════════════════════╛       │
│                                                       │
│  'água' →         ╔═══════════════════════════╗       │
│                   │ 'tipo': 'bebida'          │       │
│                   │ 'dep': 'z2'               │       │
│                   ╘═══════════════════════════╛       │
│                                                       │
│  'chocolate' →    ╔═══════════════════════════╗       │
│                   │ 'tipo': 'bebida'          │       │
│                   │ 'dep': 'z2'               │       │
│                   ╘═══════════════════════════╛       │
╘                                                       ╛
╚═══════════════════════════════════════════════════════╝
```

👣 PASSO A PASSO: Construindo dicionário de dicionários

```python
def criar_dicionario_completo(dProdTipo, dTipoDep):
    """
    Cria um dicionário onde:
    - Chave: produto
    - Valor: dicionário com 'tipo' e 'dep'
    """

    dCompleto = {}

    print("🔍 CRIANDO DICIONÁRIO DE DICIONÁRIOS...\n")

    # PASSO 1: Percorrer cada produto
    for produto, tipo in dProdTipo.items():
        print(f"📦 Processando produto: {produto}")
        print(f"   → Tipo: {tipo}")

        # PASSO 2: Buscar o depósito deste tipo
        deposito = dTipoDep.get(tipo)
        print(f"   → Depósito: {deposito}")
```

```python
        # PASSO 3: CRIAR UM DICIONÁRIO INTERNO com as informações
        info_produto = {
            'tipo': tipo,
            'dep': deposito
        }
        print(f"   → Dicionário interno criado: {info_produto}")

        # PASSO 4: Associar o produto ao dicionário interno
        dCompleto[produto] = info_produto

        print(f"   ✅ Produto '{produto}' adicionado com sucesso!")
        print("-" * 50)

    return dCompleto

# Testando
dProdTipo = {
    'álcool Gel': 'higiene',
    'água': 'bebida',
    'chocolate': 'bebida',
    'biscoito': 'alimento'
}

dTipoDep = {
    'higiene': 'z1',
    'bebida': 'z2',
    'alimento': 'z3'
}

resultado = criar_dicionario_completo(dProdTipo, dTipoDep)

print("\n🎯 RESULTADO FINAL:")
for produto, info in resultado.items():
    print(f"\n   📦 {produto}:")
    print(f"      └─ tipo: {info['tipo']}")
    print(f"      └─ dep: {info['dep']}")
```

🔍 ACESSANDO informações no dicionário de dicionários

python

```python
# Como acessar dados neste formato?
dCompleto = {
    'álcool Gel': {'tipo': 'higiene', 'dep': 'z1'},
    'água': {'tipo': 'bebida', 'dep': 'z2'}
}

print("1️⃣ ACESSO DIRETO:")
print(f"Produto: álcool Gel")
print(f"  → Tipo: {dCompleto['álcool Gel']['tipo']}")
print(f"  → Depósito: {dCompleto['álcool Gel']['dep']}")

print("\n2️⃣ ACESSO COM .get() (seguro):")
produto = 'chocolate'
info = dCompleto.get(produto, {})
if info:
    print(f"Produto: {produto}")
    print(f"  → Tipo: {info.get('tipo', 'não informado')}")
    print(f"  → Depósito: {info.get('dep', 'não informado')}")
else:
    print(f"⚠️ Produto '{produto}' não encontrado!")
```

📝 PARTE 5: EXERCÍCIOS RESOLVIDOS PASSO A PASSO

📌 EXERCÍCIO 1: Contar alunos por laboratório

python
```python
# Dados do problema
dAlunosTurmas = {
    'Ana': '33A',
    'Patinhas': '33A',
    'Zé': '33A',
    'Pedro': '33C',
    'Carla': '33B',
    'So': '33B',
    'Margarida': '33A',
    'Donald': '33B',
    'Lola': '33C',
    'Carlos': '33A',
    'Maiso': '33G'
}

dTurmasLab = {
    '33A': 'L258',
    '33B': 'L270',
    '33C': 'L314',
    '33G': 'L258'
}

print("🎯 OBJETIVO: Contar quantos alunos em cada laboratório")

# PASSO 1: Primeiro, precisamos saber QUAL laboratório cada aluno faz prova
print("\n📋 PASSO 1: Descobrir laboratório de cada aluno")
dAlunoLab = {}
for aluno, turma in dAlunosTurmas.items():
    lab = dTurmasLab.get(turma)
    dAlunoLab[aluno] = lab
    print(f"   {aluno} (turma {turma}) → laboratório {lab}")

print(f"\n📌 dAlunoLab = {dAlunoLab}")

# PASSO 2: Agora contar quantos alunos em cada laboratório
print("\n📋 PASSO 2: Contar alunos por laboratório")
dFreqLab = {}
for aluno, lab in dAlunoLab.items():
    # Usando get para contar
    dFreqLab[lab] = dFreqLab.get(lab, 0) + 1
    print(f"   Aluno {aluno} no lab {lab}: contagem agora = {dFreqLab[lab]}")

print(f"\n🎯 RESULTADO: {dFreqLab}")
print("   Isso significa:")
for lab, qt in dFreqLab.items():
    print(f"   → Lab {lab}: {qt} alunos")
```

📌 EXERCÍCIO 2: Agrupar alunos por laboratório

python
```python
print("🎯 OBJETIVO: Listar alunos de cada laboratório")
```

```python
# Usando o dAlunoLab que criamos no exercício anterior
dGruposLab = {}

print("\n📋 PASSO A PASSO:")
for aluno, lab in dAlunoLab.items():
    print(f"📥 Aluno: {aluno} → Lab: {lab}")

    # Três formas de fazer - vamos ver todas!

    # FORMA 1: Tradicional (mais fácil de entender)
    if lab not in dGruposLab:
        dGruposLab[lab] = []
        print(f"   → Criando lista para {lab}")
    dGruposLab[lab].append(aluno)
    print(f"   → Adicionando {aluno}")

    # FORMA 2: Com get (apenas para conhecimento)
    # lista = dGruposLab.get(lab, [])
    # lista.append(aluno)
    # dGruposLab[lab] = lista

    # FORMA 3: Com setdefault (mais elegante)
    # dGruposLab.setdefault(lab, []).append(aluno)

print(f"\n🎯 RESULTADO: {dGruposLab}")
print("\n📋 Lista de alunos por laboratório:")
for lab, alunos in dGruposLab.items():
    print(f"\n   {lab}:")
    for aluno in alunos:
        print(f"      └─ {aluno}")
```

📋 Tabela de Métodos

Situação	O que usar	O que faz
Contar ocorrências	`d[chave] = d.get(chave, 0) + 1`	Soma 1 a cada vez
Agrupar em listas	`d.setdefault(chave, []).append(valor)`	Adiciona item à lista
Verificar existência	`if chave in d:`	True/False
Obter com valor padrão	`d.get(chave, padrao)`	Retorna valor ou padrão

🏆 DESAFIO FINAL

```python
"""
DESAFIO: Sistema de Biblioteca

Temos 3 dicionários:
- Livros por categoria
- Categoria por prateleira
```

```
- Informações dos livros (ano, autor)

Complete as funções:
"""

# Dados
livros_categoria = {
    'Python Básico': 'programação',
    'Algoritmos': 'programação',
    'Romeu e Julieta': 'literatura',
    'Dom Casmurro': 'literatura',
    'Matemática': 'exatas'
}

categoria_prateleira = {
    'programação': 'P1',
    'literatura': 'P2',
    'exatas': 'P3'
}

livros_info = {
    'Python Básico': {'autor': 'João', 'ano': 2020},
    'Algoritmos': {'autor': 'Maria', 'ano': 2019},
    'Romeu e Julieta': {'autor': 'Shakespeare', 'ano': 1597}
}

# 1) Dicionário de Frequência: quantos livros em cada prateleira?
def contar_livros_por_prateleira(livros_categoria, categoria_prateleira):
    # Complete
    pass

# 2) Dicionário de Agrupamentos: lista de livros por prateleira
def agrupar_livros_por_prateleira(livros_categoria, categoria_prateleira):
    # Complete
    pass

# 3) Dicionário de Dicionários: livro com todas as informações
def criar_catalogo_completo(livros_categoria, categoria_prateleira, livros_info):
    # Complete
    pass

# Teste suas funções!
```

✅ GABARITO DOS MINITESTES

Miniteste 1 - Respostas

Questão 1:

python

```
pessoas = ['Ana Silva', 'João Souza', 'Maria Silva', 'Pedro Santos', 'Carlos Souza']

dFreq = {}

for pessoa in pessoas:
    partes = pessoa.split()
    sobrenome = partes[1]
    dFreq[sobrenome] = dFreq.get(sobrenome, 0) + 1

print(dFreq)  # {'Silva': 2, 'Souza': 2, 'Santos': 1}
```

Questão 2: Alternativa **b** - Existem 5 produtos no z1, 4 no z2 e 3 no z3

Miniteste 2 - Respostas

Questão 1:
python
```python
pessoas = ['Ana Silva', 'João Souza', 'Maria Silva', 'Pedro Santos', 'Carlos Souza']

dGrupos = {}

for pessoa in pessoas:
    nome_completo = pessoa.split()
    sobrenome = nome_completo[1]

    # Usando setdefault (mais elegante)
    dGrupos.setdefault(sobrenome, []).append(pessoa)

    # OU tradicional:
    # if sobrenome not in dGrupos:
    #     dGrupos[sobrenome] = []
    # dGrupos[sobrenome].append(pessoa)

print(dGrupos)
# {'Silva': ['Ana Silva', 'Maria Silva'],
#  'Souza': ['João Souza', 'Carlos Souza'],
#  'Santos': ['Pedro Santos']}
```
Questão 2: Alternativa **b** - Cria `{'grupo1': ['item1', 'item2'], 'grupo2': ['item3']}`